Project Report — Machine Learning

Reinforcement Learning in the Iterated Prisoner's Dilemma:

Convergence and Comparative Analysis of Strategies

Gabriel Tseng Axel Rubio Benjamin Krief
Krea Fares

Master 203 — Université Paris-Dauphine

Machine Learning Course — March 21, 2026

Abstract

Game theory provides a rigorous framework for analysing strategic interactions in which individual incentives diverge from collective welfare, a tension that arises naturally in multi-agent systems like financial markets. The Iterated Prisoner's Dilemma (IPD) is a canonical instance of this tension, and applying reinforcement learning to it allows us to study how adaptive agents learn to navigate without any prior knowledge of the game structure.

This report will go through the application of Q-learning to the IPD. There are two objectives: first, to understand how a rational learning agent behaves when facing classical deterministic strategies; second, to assess the relative importance of Q-learning hyperparameters : learning rate α, discount factor γ, ε-greedy exploration policy.

Our experiments show that a Q-learning agent converges to systematic defection when facing any player that do not punish defection, consistent with the dominant Nash equilibrium of the one-shot game. We compare its performance against a set of baseline strategies, ranging from naive heuristics (*Always Cooperate*, *Always Defect*) to reactive strategies (*Tit for Tat* and its variants), and discuss the conditions under which cooperation can or cannot emerge in a mixed population.

Contents

1 Introduction: The Prisoner's Dilemma **3**
1.1 Game description . . . 3
1.2 The iterated version (IPD) . . . 3
1.3 Research question . . . 4
1.4 Outline . . . 4

2 Experimental Framework and Strategy Set **5**
2.1 Simulator architecture . . . 5
2.2 Implemented strategies . . . 5
2.2.1 Naive strategies (baseline heuristics) . . . 5
2.2.2 Reactive strategies . . . 5

3 The Q-Learning Model **6**
3.1 Justification of choice . . . 6
3.2 Mathematical formulation . . . 6

4 Comparison with Baseline Strategies **7**

5 Hyperparameter Sensitivity Analysis **7**

6 Experimental Results **7**

7 Conclusion **7**

1 Introduction: The Prisoner's Dilemma

1.1 Game description

The Prisoner's Dilemma is a symmetric two-player game popularised in the context of game theory by Robert Axelrod in 1984. At each round, two players simultaneously and independently choose between two actions: **cooperate** (C) or **betray** (B). The payoff matrix is defined as follows:

Table 1: Payoff matrix of the Prisoner's Dilemma (row = player 1, column = player 2).

	Cooperate (C)	**Betray (B)**
Cooperate (C)	(3, 3)	(0, 4)
Betray (B)	(4, 0)	(1, 1)

Those payoffs are defined such that they satisfy the following inequalities (as long as they do, the game is a Prisoner's Dilemma):

$$T > R > P > S \tag{1}$$

where

- T is the temptation to defect (4)
- R is the reward for mutual cooperation (3)
- P is the punishment for mutual defection (1)
- S is the sucker's payoff (0)

The structure of this game gives rise to a fundamental tension between individual rationality and collective optimum: **betraying is a strictly dominant strategy** as it yields a higher payoff regardless of the other player's action, yet if both players choose it, the outcome $(B, B) = (1, 1)$ is Pareto-dominated by $(C, C) = (3, 3)$.

1.2 The iterated version (IPD)

In the Iterated Prisoner's Dilemma (IPD), both players interact a large number of times. In the original version, they played 200 times. Here, we set $T = 500{,}000$ rounds for convergence purposes. The repetition of the game changes the analysis: the possibility of punishing past defection and rewarding past cooperation allows conditional strategies to establish themselves. Axelrod demonstrated empirically, through computer tournaments, that the *Tit for Tat* strategy (cooperate on the first move, then mirror the opponent's previous action) achieves remarkable performance in an ecosystem of varied strategies. In fact, the strategy won in the two tournaments he organized.

1.3 Research question

This project will try to answer to the following question:

Can an agent equipped with reinforcement learning (Q-learning) learn an effective strategy in the IPD, and under what conditions does it converge to cooperation or defection?

1.4 Outline

In order to answer this question, we will first implement a Q-learning agent from scratch and confront it with a controlled set of opponents in a battle. Not taking into account any prior knowledge of the game, we observe and measure the agent's emergent behaviour : its cooperation rate, its exploitation of weak opponents, its response to punitive strategies, and compare these outcomes to the Nash equilibrium prediction. An analysis over the key hyperparameters of the model (α, γ, ε) then allows us to highlight the role of each component of the learning rule. We shall then extend the analysis to multiple Q-learning agents interacting with each other, and observe the emergence of cooperation or defection in a mixed population.

The report is organised as follows. Section 2 presents the full set of implemented strategies and the experimental framework. Section 3 describes the Q-learning model, its mathematical formulation, and the rationale for this choice. Section 4 compares baseline strategy performance. Section 5 analyses hyperparameter sensitivity. Section 6 presents our experimental results, and Section 7 concludes.

2 Experimental Framework and Strategy Set

2.1 Simulator architecture

The simulator is built around three components: a `Strategy` (the decision rule), an `Agent` (a player carrying a strategy, a score, and a local interaction memory), and a `Game` (the orchestrator). At each round, both agents simultaneously submit an action to the `Game`, which resolves payoffs from the matrix (same as in 1), updates scores, and writes the outcome to each agent's memory. All distinct agent pairs are played exhaustively for $T = 500{,}000$ rounds.

2.2 Implemented strategies

In order to test a learning agent, we implemented a set of strategies to play against.
These strategies can be divided into two categories: naive strategies and reactive strategies.

2.2.1 Naive strategies (baseline heuristics)

`AlwaysCooperate` Cooperates unconditionally at every round. Represents the maximum cooperation bound.

`AlwaysBetray` Defects systematically. Constitutes the Nash equilibrium of the IPD.

`RandomAction` Defects or cooperates with the same probability $\frac{1}{2}$.

`ProbaCooperation` Cooperates with a fixed probability p (parameter).

2.2.2 Reactive strategies

`TitForTat` (TFT) Cooperates on the first round, then mirrors the opponent's previous action.

`SuspiciousTitForTat` Identical to TFT but starts with a defection.

`TitForTwoTats` Only defects after two consecutive defections by the opponent (more lenient).

`TwoTitsForTat` Defects twice following a single received defection (more punitive).

`Joss` TFT with noise: defects randomly with probability p_j to exploit cooperators.

`DiscriminatingAltruist` Cooperates with any player who has never defected; otherwise refuses engagement.

`Bully` Defects until the opponent defects, then cooperates. Exploits accommodating strategies.

3 The Q-Learning Model

3.1 Justification of choice

We wanted to analyse the Prisoner's Dilemma from a different perspective than the one of game theory. As part of the game theory, this obvioulsy a game. As such, an agent at each state of the game will earn a reward. From a machine learning perspective, this is a reinforcement learning problem. Therefore, we could chose between different RL algorithms (SARSA, Deep Q-Network etc.). We chose Q-learning because it is a simple and efficient algorithm that can be used to solve this problem.

Indeed, SARSA is an on-policy algorithm, which onverges to a different policy under continuous exploration; it is less suitable here since we wish to evaluate the optimal policy. On the other hand, Deep Q-Network is a more complex algorithm that requires more data to train. Here, the space is minimal, so we don't need to use a neural network to approximate the Q-values.

Regarding the IDP and the Q-learning algorithm, we make the following assumptions:

- **Suitability for discrete spaces**: the IPD has a finite, discrete state space (the memory is reduced to each player's last action) making the tabular representation (q-table) efficient.
- **Off-policy**: the agent can learn the optimal policy even while exploring, which is relevant for the IPD where exploration allows testing the opponent's reactions.
- **Guaranteed convergence** under standard conditions.

3.2 Mathematical formulation

State space. The state s_t at time t is the pair of actions from the previous round.

Action space. $\mathcal{A} = \{C, B\}$.

Reward. The immediate reward r_t is the payoff matrix value obtained at round t (cf. Table 1).

Update rule (Bellman equation). The Q-table is updated after each round according to:

$$Q(s_t, a_t) \;\leftarrow\; Q(s_t, a_t) \;+\; \alpha\Big[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\Big], \tag{2}$$

where $\alpha \in (0, 1]$ is the learning rate and $\gamma \in [0, 1)$ the discount factor (valuation of future rewards).

ε-greedy policy. The agent selects a random action with probability ε (exploration) and the action with the highest Q-value with probability $1 - \varepsilon$ (exploitation). ε decays multiplicatively at each step by a factor `epsilon_decay`, down to a floor `epsilon_min`. We could use a constant epsilon, but the goal is to learn the optimal policy, so we need to explore at the beginning and then exploit.

4 Comparison with Baseline Strategies

5 Hyperparameter Sensitivity Analysis

6 Experimental Results

7 Conclusion